

Arrays

Conjuntos de datos

```
1 let listaDeNumeros = [2, 3, 5, 7, 11];  
2 console.log(listaDeNumeros[2]);  
3 // → 5  
4 console.log(listaDeNumeros[0]);  
5 // → 2  
6 console.log(listaDeNumeros[2 - 1]);  
7 // → 3
```

Para trabajar con un conjunto de datos digitales, primero tenemos que encontrar una forma de representarlo en la memoria de nuestra máquina. Digamos, por ejemplo, que queremos representar una colección de los números 2, 3, 5, 7 y 11.

Podríamos ser creativos con las cadenas, después de todo, las cadenas pueden tener cualquier longitud, por lo que podemos poner muchos datos en ellas, y usar "2 3 5 7 11" como nuestra representación. Pero esto es incómodo. Tendríamos que extraer de alguna manera los dígitos y convertirlos de vuelta a números para acceder a ellos.

Afortunadamente, JavaScript proporciona un tipo de dato específicamente para almacenar secuencias de valores. Se llama un *array* y se escribe como una lista de valores entre corchetes, separados por comas:

Acceso a elementos del array

La notación para acceder a los elementos dentro de un array también utiliza corchetes. Un par de corchetes inmediatamente después de una expresión, con otra expresión dentro de ellos, buscará el elemento en la expresión de la izquierda que corresponde al *índice* dado por la expresión en los corchetes.

El primer índice de un array es cero, no uno, por lo que el primer elemento se recupera con `listaDeNumeros[0]`. El conteo basado en cero tiene una larga tradición en tecnología y, de cierta manera, tiene mucho sentido, pero se necesita un poco de tiempo para acostumbrarse. Piensa en el índice como el número de elementos a omitir, contando desde el inicio del array.

Métodos

Tanto los valores de cadena como los de array contienen, además de la propiedad `length`, varias propiedades que contienen valores de función.

```
let doh = "Doh";
console.log(typeof doh.toUpperCase);
// → function
console.log(doh.toUpperCase());
// → DOH
```

Cada cadena de texto tiene una propiedad `toUpperCase`. Cuando se llama, devolverá una copia de la cadena en la que todas las letras se han convertido a mayúsculas. También existe `toLowerCase`, que hace lo contrario.

Curiosamente, aunque la llamada a `toUpperCase` no pasa argumentos, de alguna manera la función tiene acceso a la cadena "Doh", el valor cuya propiedad llamamos.

Las propiedades que contienen funciones generalmente se llaman *métodos* del valor al que pertenecen, como en "toUpperCase es un método de una cadena".

Este ejemplo demuestra dos métodos que puedes utilizar para manipular arrays:

```
let secuencia = [1, 2, 3];
secuencia.push(4);
secuencia.push(5);
console.log(secuencia);
// → [1, 2, 3, 4, 5]
console.log(secuencia.pop());
// → 5
console.log(secuencia);
// → [1, 2, 3, 4]
```

El método `push` agrega valores al final de un array. El método `pop` hace lo opuesto, eliminando el último valor en el array y devolviéndolo.

Estos nombres son términos tradicionales para operaciones en una *pila*. Una pila, en programación, es una estructura de datos que te permite agregar valores a ella y sacarlos en el orden opuesto para que lo que se agregó último se elimine primero. Las pilas son comunes en programación;

Estructura FOR.. OF

Esta variación del bucle for estándar permite iterar un objeto a través de sus valores enumerables. Los objetos iterables por esta sentencia pueden ser matrices, mapas, argumentos, conjuntos de datos, ... Por cada propiedad que se captura, JavaScript ejecuta su contenido hasta que ya no haya más valores que capturar.

```
let arr = [1, 1, 2, 3, 5, 8];  
for (let x of arr){  
    console.log("X: ", x); // Devuelve 1,1,2,3,5,8  
}
```

Aunque esta forma de recorrer los objetos pueda resultar muy cómoda, no es recomendable utilizarla cuando el número de elementos es muy elevado o cuando el objeto a recorrer es muy grande en tamaño porque el rendimiento puede bajar de forma abrumadora.

La sentencia **for...of** ejecuta un bucle que opera sobre una secuencia de valores provenientes de un **objeto iterable**.



(for...of)

Sintaxis

```
for (variable of iterable)  
  statement
```

Parámetros del bucle

variable

Recibe un valor de la secuencia en cada iteración. Puede ser una declaración con const, let, o var, o un objetivo de asignación (p. ej., una variable previamente declarada, una propiedad de objeto o un patrón de asignación por desestructuración). Las variables declaradas con var no son locales al bucle, es decir, están en el mismo ámbito en el que se encuentra el bucle for...of.

Componentes adicionales

iterable

Un objeto iterable. La fuente de la secuencia de valores sobre la que opera el bucle.

statement

Una sentencia que se ejecutará en cada iteración. Puede hacer referencia a variable. Puedes usar una **sentencia de bloque** para ejecutar múltiples sentencias.

Un bucle for...of opera sobre los valores provenientes de un iterable, uno por uno y en orden secuencial. Cada operación del bucle sobre un valor se denomina *iteración*, y se dice que el bucle *itera sobre el iterable*. Cada iteración ejecuta sentencias que pueden referirse al valor actual de la secuencia.